

the proof of Lemma 7.3 that $V_{p,k,f}$ is finite, namely $\{X^i Y^j : 0 \leq i, j < p^k - 1\}$ generates $V_{p,k,f}$ as an $\mathbb{F}_p$ -module. Then its universal p^k -ring quotient $U_{p,k,f} := (V_{p,k,f})_{[p^k]}$ from Definition 3.7 does the job: If two units a, b of a p^k -ring with $p = 0$ satisfy $ba = f(a)b$, then there is a unique homomorphism of rings $U_{p,k,f} \rightarrow A$ that maps $X \mapsto a$ and $Y \mapsto b$. Thus, if $U_{p,k,f}$ commutative, we have $ab = ba$. $\square$

Remark 8.14. With the notation above, $\{X^i Y^j : 0 \leq i, j < p^k - 1\}$ is not necessarily an $\mathbb{F}_p$ -basis of $V_{p,k,f}$. For example, $X = 1$ holds in $V_{2,3,T^2}$. More generally, when p, k are arbitrary and $f = T^m$ is any monomial (which is sufficient by Theorem 7.9), then we have $X^d = 1$ for $d := \gcd(m^{p^k-1} - 1, p^k - 1)$, and we see that $V_{p,k,f} = \mathbb{F}_p[G]$ is a group algebra for

$$G := \langle X, Y : X^d = 1, Y^{p^k-1} = 1, YXY^{-1} = X^m \rangle \cong C_d \rtimes_m C_{p^k-1}.$$

This description should help to calculate $U_{p,k,T^m} = \mathbb{F}_p[G]_{[p^k]}$, but in practice a brute force attack is still too inefficient, even for small numbers.

Appendix A. SageMath code

A.1. Computation of n -primes and n -powers.

The code below has been used to generate the list of n -fields in Remark 3.1.

```
def n_primes(n):
    """Returns the list of prime numbers p with p-1 | n-1"""
    return [x+1 for x in divisors(n-1) if (x+1).is_prime()]

def n_powers(n):
    """Returns the list of prime powers q with q-1 | n-1"""
    return [x+1 for x in divisors(n-1) if (x+1).is_prime_power()]

def all_powers(limit):
    """Returns the lists of n-powers for 2 <= n <= limit"""
    return [[n, n_powers(n)] for n in range(2, limit+1)]
```

A.2. Reduction to prime characteristic.

The code below implements the algorithm in Lemma 3.11.

```
def get_numbers_for_characteristic(n):
    """Returns a list of numbers z such that p_1 * ... * p_s is the gcd of
    the z^n - z, where p_i are the prime numbers with p_i - 1 | n - 1"""
    c = prod(n_primes(n))
    z, g, numbers = 0, 0, []
    while (g != c):
        z = z.next_prime()
        numbers.append(z)
        g = gcd(g, z^n - z)
    return numbers

def characteristic_proof(n):
    """Returns an equational proof that p_1 * ... * p_s = 0 holds in
    any n-ring, where p_i are the prime numbers with p_i - 1 | n - 1"""
    numbers = get_numbers_for_characteristic(n)
    myprimes = n_primes(n)
    relations_string = ", ".join([f"{z}^{{n}}-{z}" for z in numbers])
    factor_string = " * ".join(map(str, myprimes))
    return f"gcd({relations_string}) = {str(prod(myprimes))} = {factor_string}"
```

To get the linear combination, one can use the `extended_gcd` function below.

A.3. Reduction to prime powers.

The code below implements the reduction algorithm from Remark 4.3. For example, the command `reduction_proof(7, 2)` returns a proof that every 7-ring with $2 = 0$ is a 4-ring.

```
def extended_gcd(args):
    """Extension of Sage's xgcd function from two to several arguments.
    Given args = [a_1, ..., a_n], returns [gcd(a_1, ..., a_n), [u_1, ..., u_n]]
    where u_1 * a_1 + ... + u_n * a_n = gcd(a_1, ..., a_n)."""
    if (len(args) == 0):
        return [0, []]
    if (len(args) == 1):
        return [args[0], [1]]
    if (len(args) == 2):
        g, u, v = xgcd(*args)
        return [g, [u, v]]
    first, *rest = args
    rest_gcd, v = extended_gcd(rest)
    full_gcd, u = extended_gcd([first, rest_gcd])
    w = [u[0]] + [u[1] * x for x in v]
    return [full_gcd, w]

def get_exponent(p, n):
    """Returns the least common multiple of all d > 0 with p^d-1 | n-1"""
    limit = floor(log(n, p))
    powers = [d for d in range(1, limit+1) if (p^d-1).divides(n-1)]
    return lcm(powers)

def reduction_proof(n, p, only_polynomials=False):
    """Returns a linear combination of the form g = sum_i u_i * (f_i^n - f_i)
    in GF(p)[T], where g divides T^(p^k) - T, thus proving the reduction theorem.
    When the flag 'only_polynomials' is set, returns only the list of the f_i."""
    if not (p.is_prime() and (p-1).divides(n-1)):
        raise Exception("Invalid arguments")

    k = get_exponent(p, n)
    R.<T> = PolynomialRing(GF(p))
    polynomials = []
    g = 0
    target = T^(p^k) - T
    found = False

    # compute gcd of polynomials of the form f^n - f until it divides T^(p^k) - T
    for f in R.polynomials(max_degree = n-1):
        if f.degree() < 1 or (g != 0 and g.divides(f^n-f)):
            continue
        polynomials.append(f)
        g = gcd(g, f^n-f)
        if (g.divides(target)):
            found = True
            break
    if not found:
        raise Exception("No linear combination found")

    if (only_polynomials):
        return polynomials
```

```

# print the linear combination
proof = str(g)
coeffs = extended_gcd([f^n-f for f in polynomials])[1]
for i in range(len(polynomials)):
    prefix = " +" if i > 0 else " ="
    coeff = coeffs[i]
    poly = polynomials[i]
    proof += f"{prefix} ({coeff}) * ({poly})^n - ({poly})"
return proof

```

A.4. Finding unpleasant numbers.

The code below has been used in Remark 4.17 to demonstrate how many simple numbers are nice. We use the `get_exponent` function from above.

```

def is_simple_at(n, p):
    """Checks if n is simple at p"""
    return n>1 and (p-1).divides(n-1) and get_exponent(p,n) == 1

def unpleasant_numbers(p,start,end):
    """Returns the number of p-simple numbers within a range and the list of
    unpleasant numbers among them"""
    R.<T> = PolynomialRing(GF(p))
    unpleasants = []
    simple_count = 0
    for n in range(start,end+1):
        if not (is_simple_at(n,p)):
            continue
        simple_count += 1
        g = gcd([(T+u)^n - (T+u) for u in GF(p)])
        if (g != T^p - T):
            unpleasants.append(n)
    return [simple_count, unpleasants]

```

A.5. Simple numbers.

The code below generates the first simple numbers as in Definition 5.1.

```

def is_simple(n):
    """Checks if n is a simple number"""
    return n>1 and all(q.is_prime() for q in n_powers(n))

def first_simple_numbers(limit):
    """Returns the list of simple numbers below a given limit"""
    return [n for n in range(2,limit+1) if is_simple(n)]

```

References

- [AD20] D. D. Anderson, P. V. Danchev, *A note on a theorem of Jacobson related to periodic rings*, Proc. Am. Math. Soc. 148.12 (2020), 5087–5089.
- [Bi35] G. Birkhoff, *On the structure of abstract algebras*, Proc. Cambridge Philos. Soc. 31 (1935), 433–454.
- [BM12] S. M. Buckley, D. MacHale, *Centriflers and ring commutativity*. J. Math. Sci. Adv. Appl. 15 (2012), 139–161.
- [BM13] S. M. Buckley, D. MacHale, *Variations on a theme: rings satisfying $x^3 = x$ are commutative*, Amer. Math. Monthly 120.5 (2013), 430–440.
- [Da01] D. van Dalen, *Intuitionistic Logic*, in Goble, Lou, ed., The Blackwell Guide to Philosophical Logic. Blackwell (2001).
- [Do76] S. W. Dolan, *A proof of Jacobson's Theorem*, Canad. Math. Bull. 19.1 (1976), 59–61.